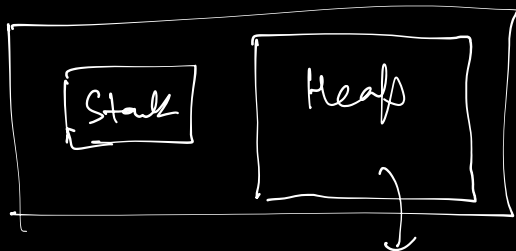


* Strings

(Stack / Heap)

RAM



Data Structure
(Stack)
is diff.

- Stack
- Less space
 - Fast
- Heap
- More space / Memory
 - Relatively slower than stack

→ object is created
↓
inside Heap

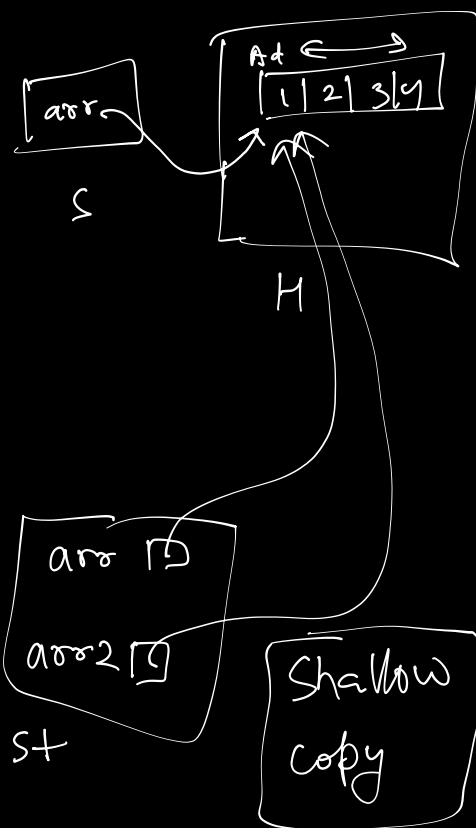
let arr = [1, 2, 3, 4]

* while writin code
you don't have access to heap

let arr2 = arr;

arr[0] = 5

↳ this is not possible in strings.

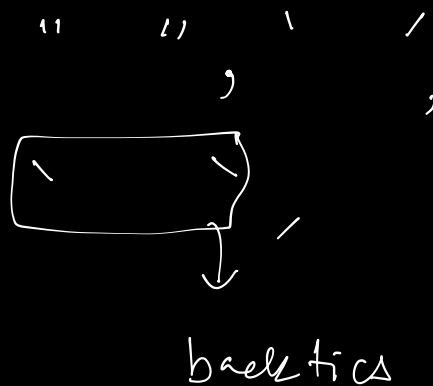


Strings

↳ Sequence of character inside

Let $s = \text{"Hello"}$
 $s = \text{'Hello'}$ → Same

$s = \text{'Hello'}$ → String Interpolation

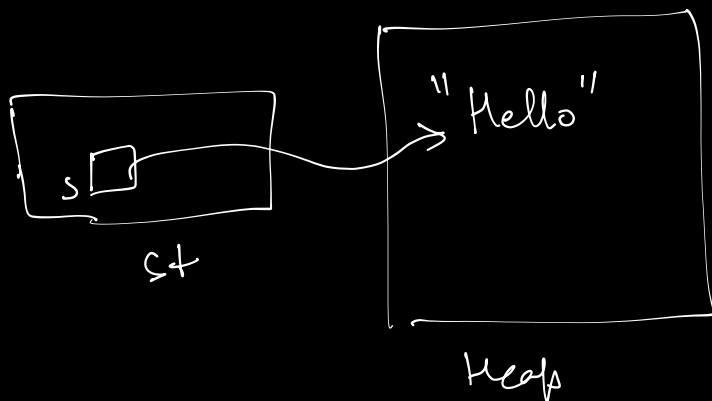


→ treat it similar to array

$s = \text{'H e l l o'}$
 ↑
 0 1 2 3 4

$s[1]$	$s[2]$	$s[0]$
↓	↓	↓
e	l	H

String are immutable



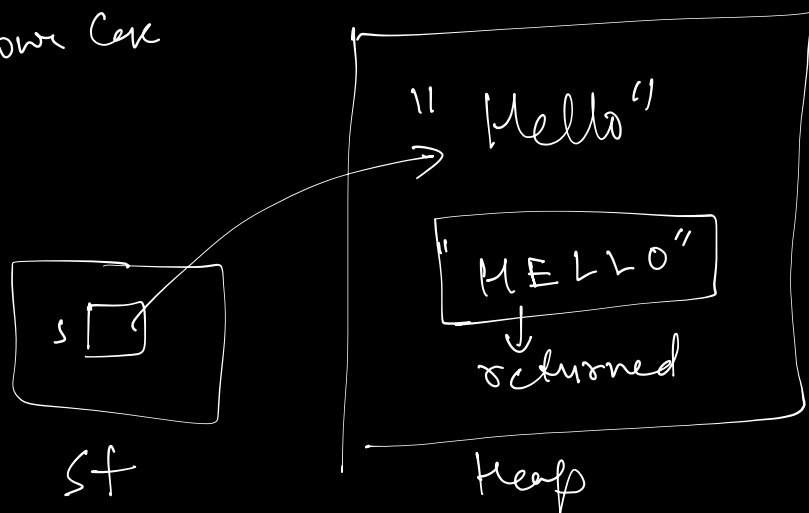
Q) String is primitive, but stored in heap. why?

length = s.length; ✓

* functions in strings

① to Upper case / to lower case

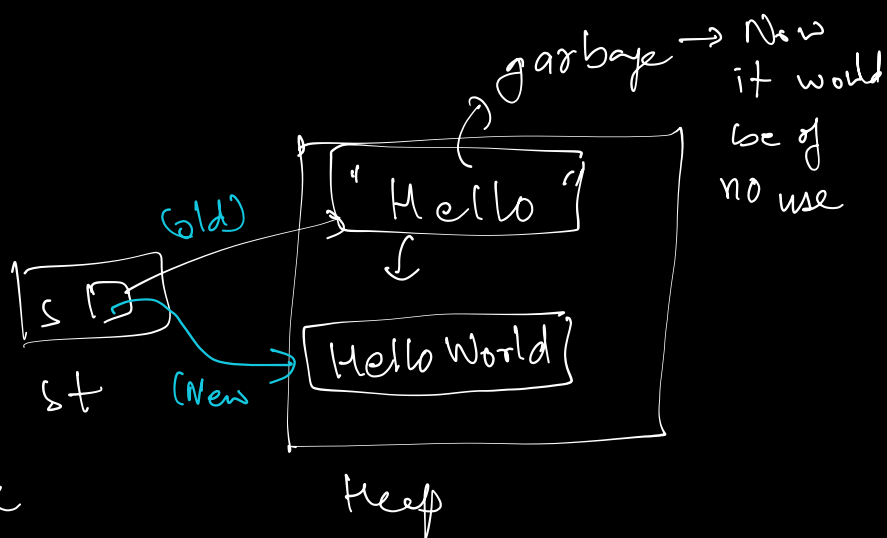
Implement



② Concatenation

`s = "Hello"`

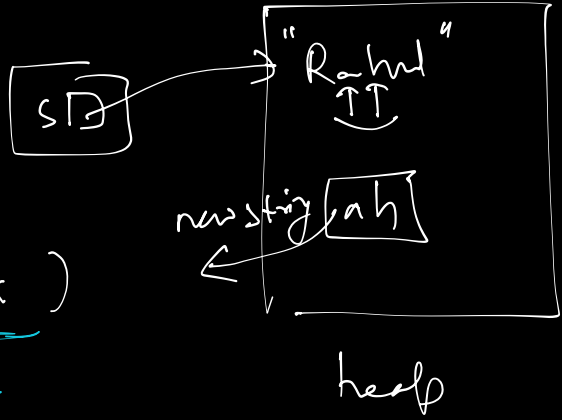
`s = s + "World"`



→ string is immutable

③ slice

S = "Rahm"
 0 1 2 3 4

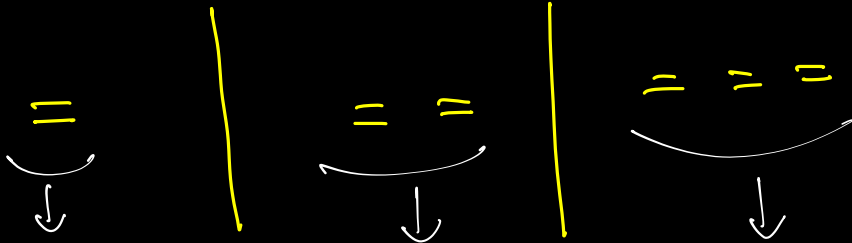


s.slice(start idx, last idx)
 ↓ ↓
 include exclude

(1, 3) ⇒ ah
 1 2 3
 ✓ ✓ ✗

* ('S' == 5) → T/F

'S' == 5
 ↓
 T/F
 ↓
 F

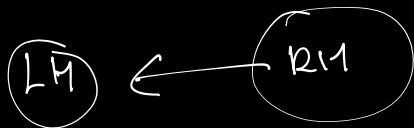


Assigning

Loosely
check

Tightly
check

↓
 Type is also checked



* String to Character Array

S = "Rohan"

```
// Problem  
// you are given a string, you have to return  
// a new string where vowels are replaced by its successive consonant
```

String $\rightarrow$ Array

t = []

t[i] = S[i]

a $\rightarrow$ b
e $\rightarrow$ f
i $\rightarrow$ j
o $\rightarrow$ p
u $\rightarrow$ v

if (t[i] == 'a')

t[i] = 'b'

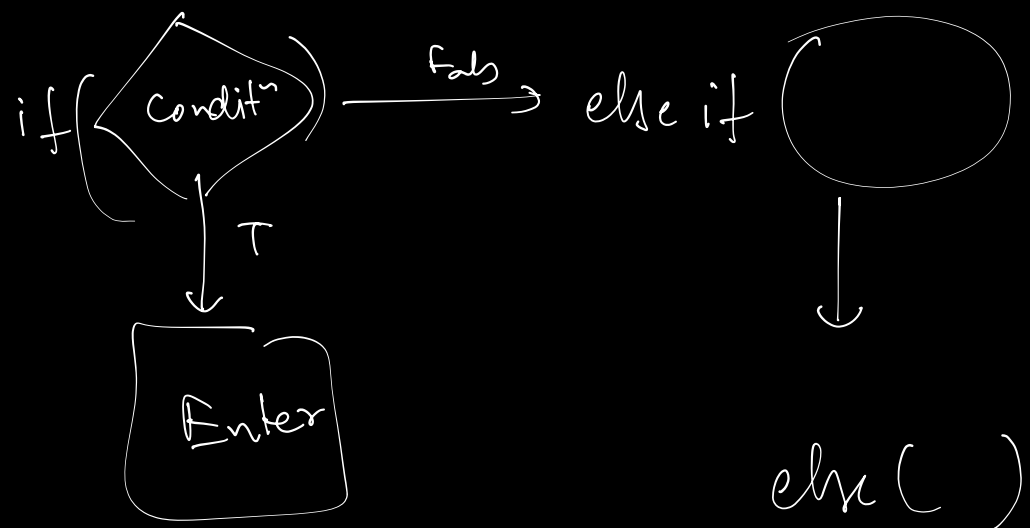
else if (== 'e')

i
o
u

S = "Rohan"

t = ['r', 'p', 'h', 'b', 'n']

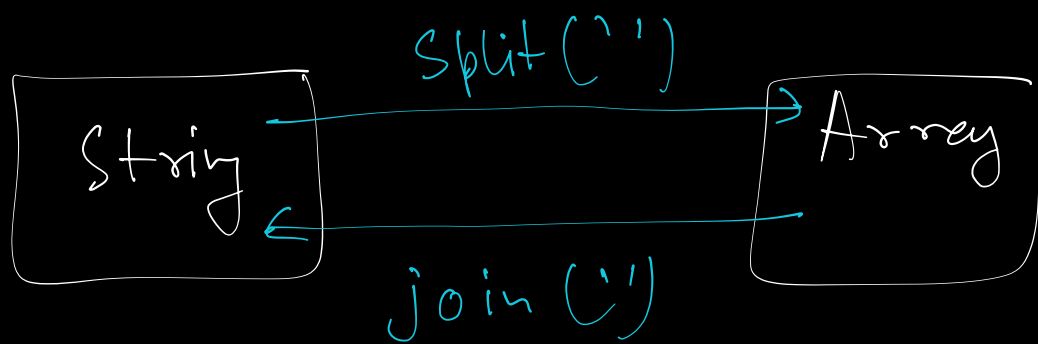
if - else if - else



s.split(' ') ('a')

0	1	2	3	4
r	o	h	a	n

"roh", "n"



* Anagram $\rightarrow$ MW

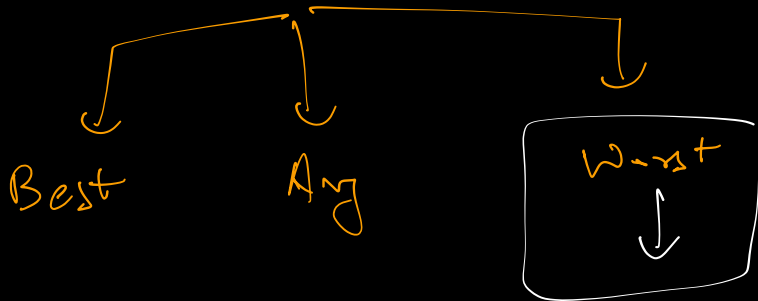
* Array of String

arr = $\begin{bmatrix} \text{"0 1 2"} \\ \text{"ra j"} \\ \text{"0 1 2 3 4"} \\ \text{"sa yan"} \\ \text{"0 1 2 3 4"} \\ \text{"ro hit"} \end{bmatrix}$

arr[1][3] $\Rightarrow$ 'a'

* Time Complexity

→ cnt



→ Big O

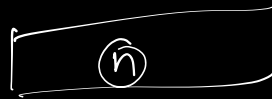
Q what is the complexity of searching an element in an array?

↳ $O(n)$

Big O Notation	$\neq$	Worst Case
-------------------	--------	---------------

↳ but most of the time it is similar.

* Space Complexity (SC)



$$TC = O(1)$$

$$SC = O(1)$$

How much memory
does a program take to run
the code.

$$SC \rightarrow f(n) \rightarrow O(1)$$

* Creating an Array of size (n) using Constructor Method

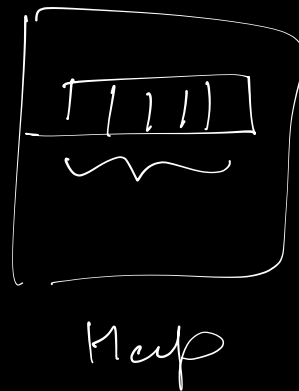
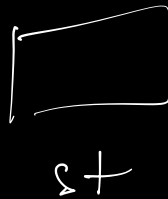
let a = new Array(n)

n : size of the i/p given to you

→ Extra memory except the given array



Auxiliary Space



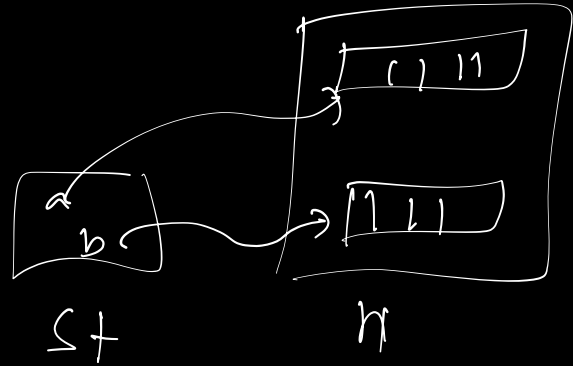
$$f(n) = n$$

$$SC = O(n)$$

let a = new Array(n);
let b = new Array(n);

$$f(n) = \overset{a}{\downarrow} n + \overset{b}{\downarrow} n = 2n$$

$$SC = O(n)$$



```
// Problem
// you are given a string, you have to return
// a new string where vowels are replaced by its successive consonant

let s = "rohan";
let z = s.split('');
let t = [];

for(let i=0; i<s.length; i++){
  t[i] = s[i];
  if(t[i] === 'a'){
    t[i] = 'b';
  }
  else if(t[i] === 'o'){
    t[i] = 'p';
  }
}

let ans = t.join('');
console.log(ans);
```

$$f(n) = \overset{t}{\downarrow} n + \overset{t}{\downarrow} n + \overset{ans}{\downarrow} n = 3n = O(n)$$

Q

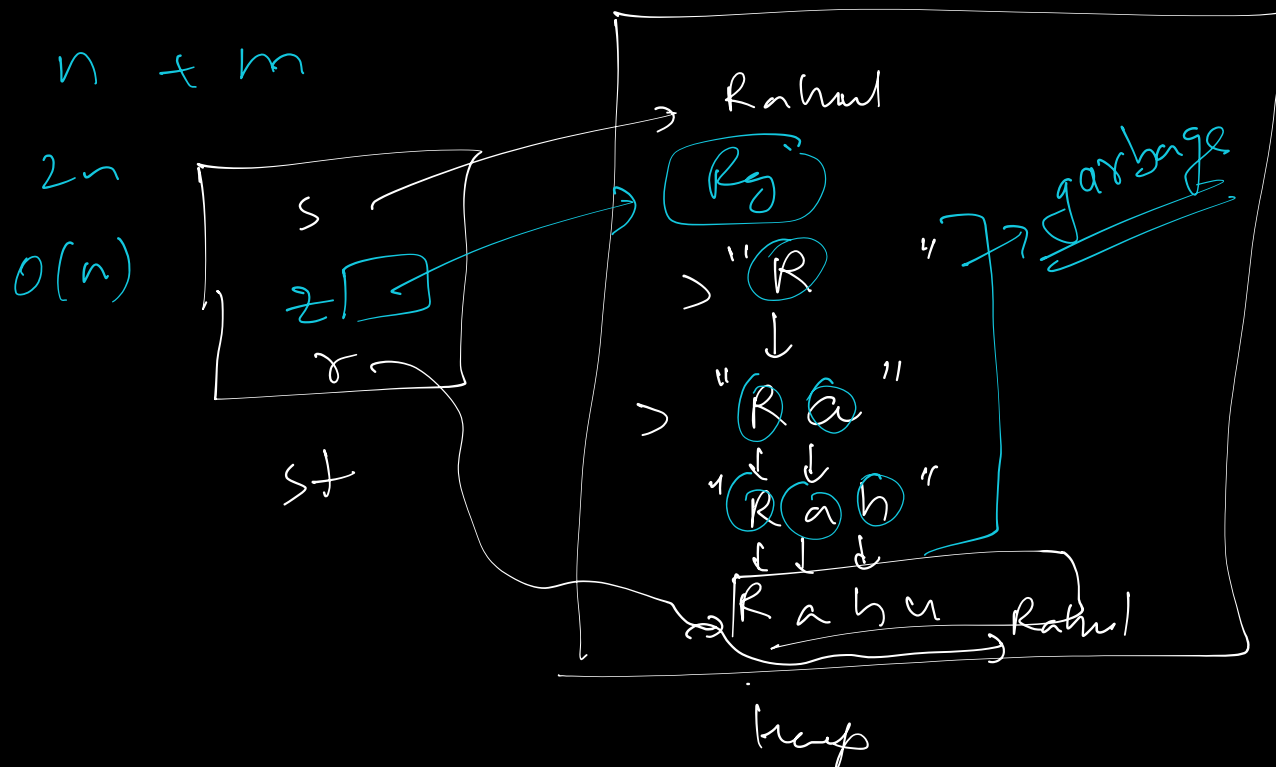
```
solve ( arr ) {  
    n ← arr.length  
    b = new Array(n * n);  
}
```

} → $O(n^2)$

```
82  
83 let str = "rahul";  
84 let result = "";  
85 for (let i = 0; i < str.length; i++) {  
86     let ch = str[i];  
87     if (ch === 'a') result += 'b';  
88     else if (ch === 'e') result += 'f';  
89     else if (ch === 'i') result += 'j';  
90     else if (ch === 'o') result += 'p';  
91     else if (ch === 'u') result += 'v';  
92     else result += ch;  
93 }  
94 console.log(result);  
95
```

$T_C = O(n^2)$

$SC = O(n)$



$$i=0$$

$$x += 'R' \rightarrow 1$$

$$i=1$$

$$x += 'a' \rightarrow 2$$

$$i=2$$

$$x += 'h' \rightarrow 3$$

$$i=3$$

$$x += 'a' \rightarrow 4$$

$$i=4$$

$$x += 'l' \rightarrow 5$$

$$TC = 1 + 2 + 3 + \dots + n-1$$

$$= \frac{n(n-1)}{2} = \frac{n^2}{2} - \frac{n}{2}$$

$$\frac{n^2}{2} \rightarrow \underline{\underline{O(n^2)}}$$

C/C++

```
int a = 5;
string s = "Rohan";
```

7B

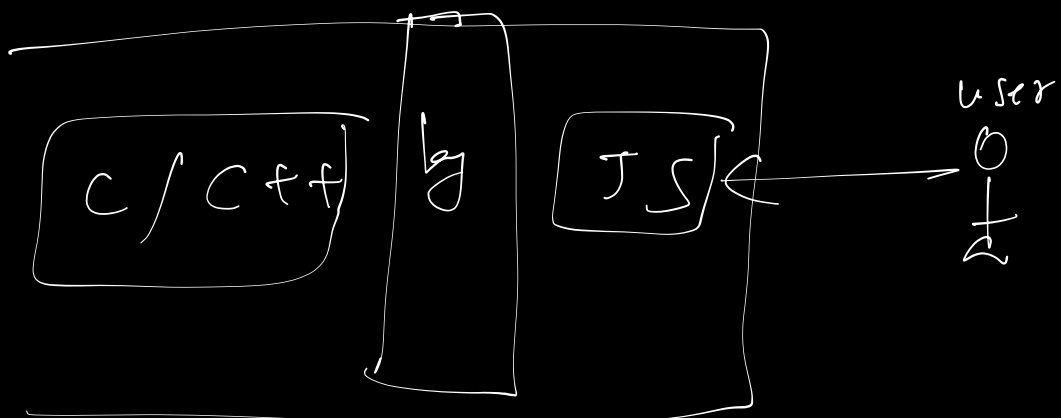
6

JavaScript

```
let a = 5;
let s = "Rohan";
```

Num

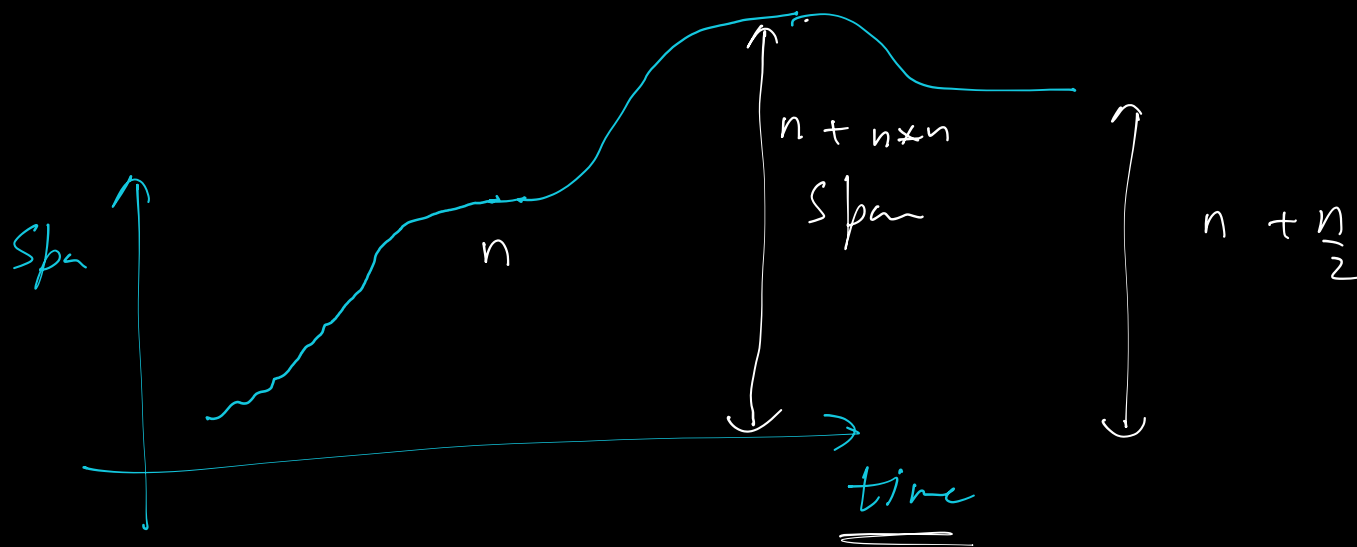
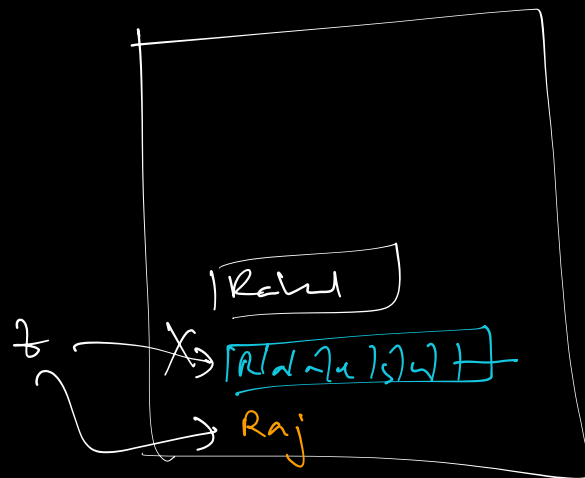
String



$x = \text{"Rahul"} (n)$

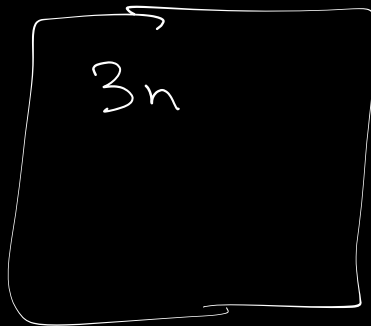
$z = \text{"Rameshvaran"} (n \times n)$

$z = \text{"Raj"} (\frac{n}{2})$



$$\text{Space Comp} = n + n \times n = \underline{\underline{O(n^2)}}$$

Run
←
[n]

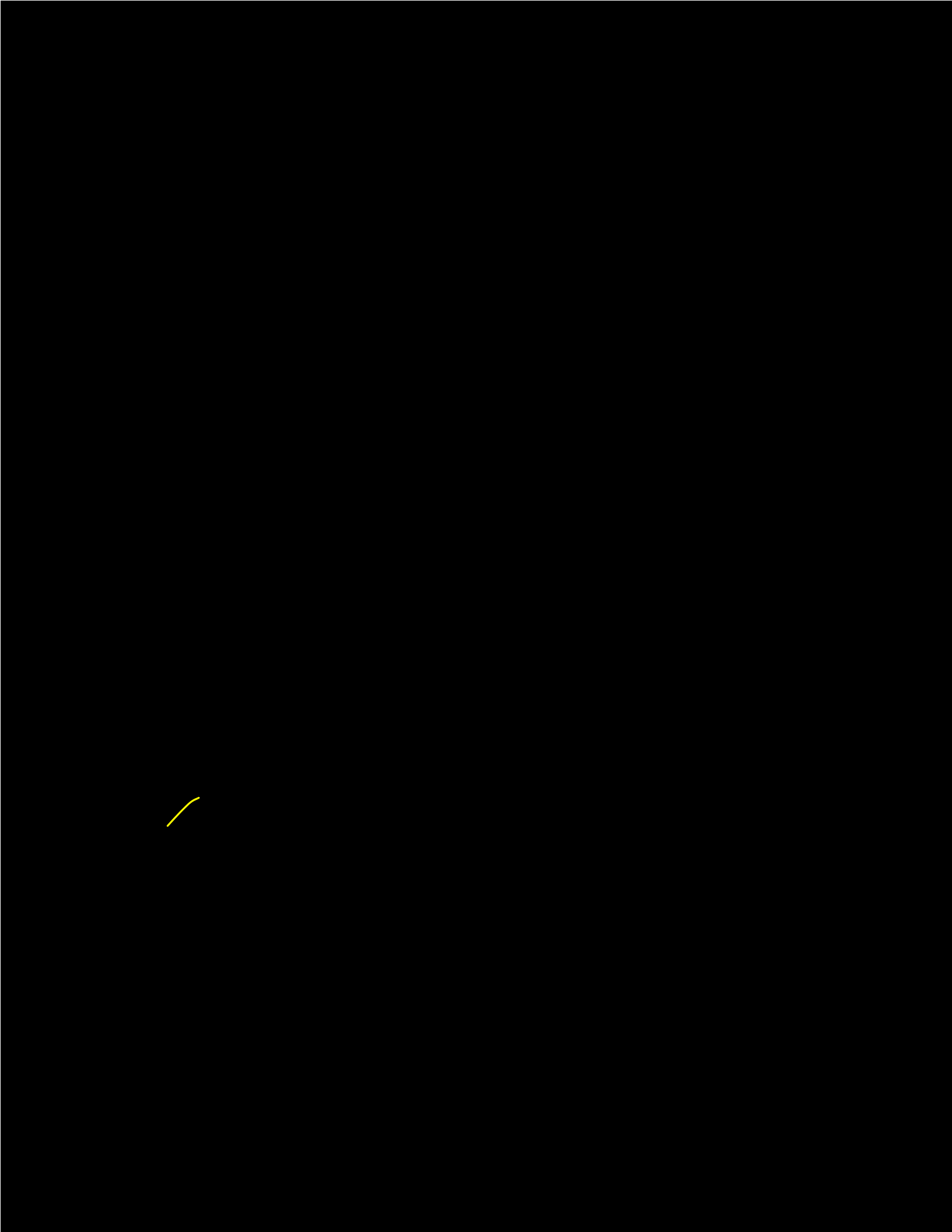


→ $n + n \times n > \underline{\underline{3n}}$

$n + \frac{n}{2} < 3n$

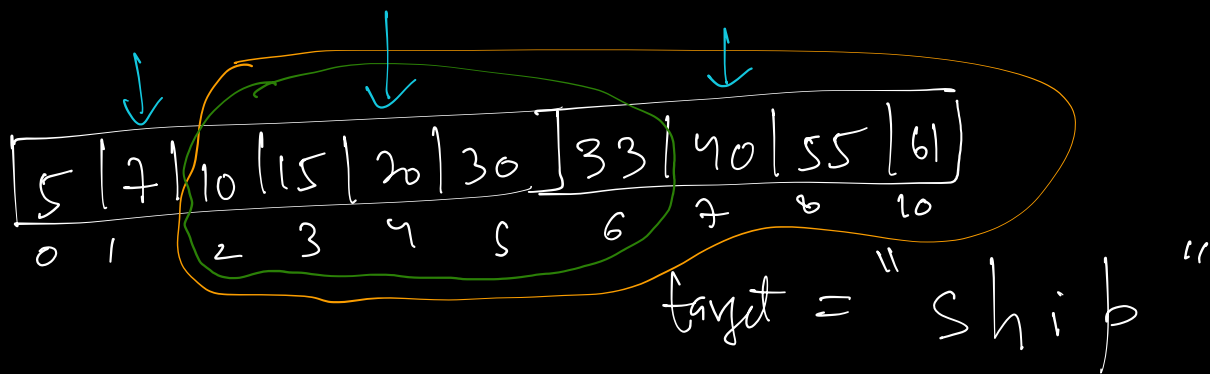
SC
⇓

Maximum Memory taken by
the program to run



Binary Search

target = 20



en sat

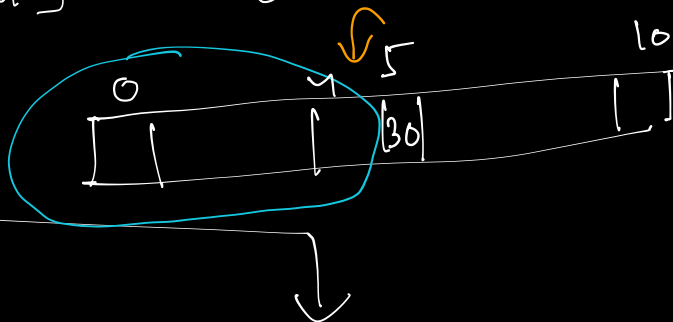
Search space/interval

target 20

① $\left[\begin{array}{l} \underline{l=0} \quad \quad \quad \underline{h=10} \quad \quad \quad \underline{\text{mid} = \frac{(0+10)}{2} = 5} \end{array} \right]$ $(n-1)$

key = arr[mid] = 30

key $\neq$ target



target < key

target > key

0 - 4

$h = \text{mid} - 1$

$\Rightarrow l=0, h=4$

② $l=0$ $h=4$ $mid = \left(\frac{0+4}{2} \right) = 2$

$key = arr[mid] = arr[2] = 10$

$target = 20$

$key \neq target$ $(10 < 20)$

$\downarrow$ $\downarrow$

$target < key$ $key < target$

$0 - 4$
 $2 \quad \text{---}$
 $\times$

$[3, 4]$

$l = mid + 1$

$l=3$ $h=4$

Maths. floor()

③ $l=3$ $h=4$ $mid = \left(\frac{3+4}{2} \right) = \lfloor 3.5 \rfloor = 3$

$key = arr[mid] = arr[3] = 15$ | $target = 20$

$key < target$

$l = mid + 1$

$l=4$ $h=4$

$\textcircled{4} \quad l = 4 \quad h = 4 \quad \text{mid} = \left(\frac{4+4}{2} \right) = 4$

$\text{key} = \text{arr}[4] = 20$

$\text{key} == \text{target}$

$\rightarrow \text{mid} \rightarrow \text{answer}$

$\leftarrow \text{Assume arr}[4] = 22$

$l = 4$

$h = 4$

$\text{mid} = 4$

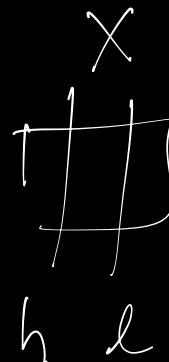
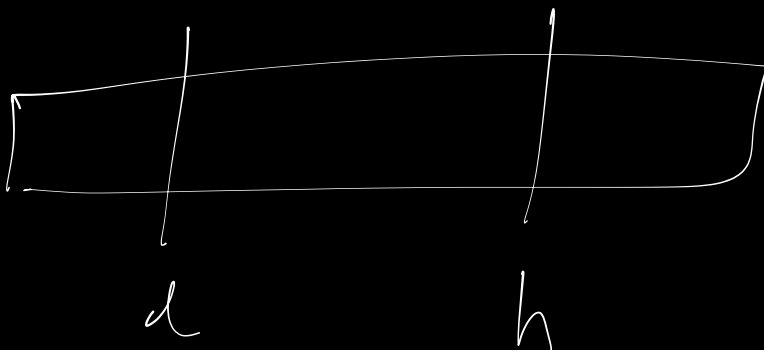
$\text{key} = \text{arr}[4] = 22$

$\text{target} < \text{key}$

$h = \text{mid} - 1$
 $= 4 - 1$

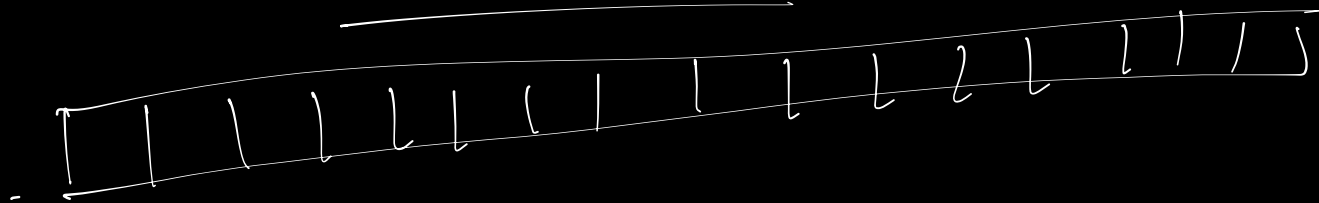
$l = 4$

$h = 3$



$l > h \Rightarrow$ we should stop

return -1



①

②

③

④

... ⑫

$$f = \underbrace{1 + 1 + 1 + \dots + 1}_K$$

$$= K \times 1$$

$$= \underline{\underline{K}}$$

K in terms of n



HW

Assign

1	7	20	21	30	40	62	70
0	1	2	3	4	5	6	7

target = 62

target = 40

target = 30